The Brilliant Employee · 08

The machine passed it. I rejected the work.

Every task my terminal AI finishes writes one line into a log file it cannot edit. Here is one real line from 3 August, taken apart field by field: what the model picker recommended, what actually ran, and whether I kept the work.

sgnk.ai/brilliant ↗

Sagnik Mitra

Why this exists

A language model writes a large share of my production code. Wrong work reads exactly like right work, and no prompt fixes that. So the system below exists to find out when the answers were wrong.

From one week, none of it visible from inside the tool: a sizing step ignored on 796 of 816 tasks, and two fixes written up as done that were never made.

THE SYSTEM, AS OF 7 AUGUST 2026, 22:55Z	
rules written	11 June, 57 days ago
ledger running	27 June, 41 days
tasks in the ledger	3,140, across 3,680 rows
controls on tool calls	**6 registered, 4 that can refuse**

You can rent the model. You cannot rent the part that tells you it was wrong.

Five parts, and where today sits

Five parts sit around the model. Each post takes one of them.

sizing check	sizes a job before a model is picked
model picker	recommends which model gets the job
safety stops	refuse the irreversible, before it runs
grader	grades finished work pass or fail
task log	at least one append-only line per task

Listed in the order they act on a task, not the order they were built.

Today is about the task log, marked above.

> **ASIDE.** Only the safety stops can refuse. The other four observe: one advises, one recommends, one grades after the fact, one records.

This page is the map. Nothing here needs another post to make sense.

The AI cannot edit its own task log

The task log is the part of the workplace that remembers. Written rule #31: every task appends ONE line to a dated file, with a correlation id and an accepted flag. Every task, not some.

```
if [ -s "$TRACE_FILE" ]; then
  printf '\n%s' "$_ROW" >> "$TRACE_FILE"
else
  printf '%s' "$_ROW" >> "$TRACE_FILE"
fi
```

My terminal AI writes that line through a hook it cannot touch and cannot revise afterwards. A worker that grades its own homework will also tidy its own transcript.

> **Plain words: THE LEDGER.** ~/.sgnk/traces/YYYY-MM-DD.jsonl. One file per day, at least one JSON line per task, written by the harness, never by the worker.

If the record can be edited by the thing being measured, it is not a record.

A diagram of a loop is not a loop that runs

Written rule #32: the words self-improving are banned in this workspace unless a grading loop is RUNNING and feeding the model picker.

- Every finished task appends one line
  `the harness writes it, not the worker`
- A grading skill scores that work
  `machine check, pass or fail`
- Those grades feed the model picker — never built
  `the arrow on the whiteboard`

Rule #32 unlocks the phrase only when that last step provably runs.

The failure mode is seductive. You build the ledger, you build the grader, you draw the arrow between them on a whiteboard, and the sentence 'the system learns from its mistakes' starts to feel true. The arrow is the part that never gets built.

> **ASIDE.** The ban applies to me as much as to it. I wrote it after catching myself about to claim the loop was closed when it was not.

A learning claim is only as real as the pipeline that consumes the data.

What one finished task looks like

Pulled from the live file for 2026-08-03, redacted only where it names the session. One task: a generate-shaped job, floor tier, single shot, in my own Macaulay2 web project at 00:12 UTC.

~/.sgnk/traces/2026-08-03.jsonl (one row, redacted)


```
{"timestamp":"2026-08-03T00:12:32Z",
"correlation_id":"SESSION:16",
"gate_tier":"floor","gate_orchestrate":"single",
"decided_model":"sonnet","decision_real_n":0,
"decision_mode":"advisory",
"task_shape":"generate","tier":"opus",
"model":"claude-opus-5",
"input_tokens":838575,"fresh_input_tokens":2,
"cache_read_tokens":838573,"output_tokens":9896,
"latency_ms":191000,
"assertion_pass":true,"assertion_n":1,
"assertion_source":"machine-gate",
"accepted":false}
```


ASIDE. Trimmed to fit the slide. The full row carries 33 keys.

One finished task, every field it wrote, appended the moment it ended.

The cheap model was picked. The big one ran.

ROUTING FIELDS	
correlation_id	SESSION:16 . the join key every downstream loop keys on
gate_tier	floor . the sizing check, the part that sizes the job, sent this to the cheap tier
decided_model	sonnet . the model picker, the part that recommends the model, recommended cheap
decision_mode	**advisory . the pick was logged, not enforced. The big model ran anyway**
decision_real_n	**0 . real observations behind that recommendation: none**
The row keeps the gap between recommendation and reality.	

Both parts spoke, and the line preserves what each said next to what actually happened. Decided sonnet, ran opus. That gap is signal you can only collect by writing both halves down.

Log the recommendation AND the outcome.
The delta between them is the signal.

Two verdicts, one line, joined forever

COST AND VERDICTS	
input_tokens	838,575 . of which 838,573 came from cache. Fresh tokens: 2
output_tokens	9,896
latency_ms	191,000 . three minutes eleven seconds of work
assertion_pass	**true . the machine check passed it, 1 of 1**
accepted	**false . I rejected the work anyway**
Machine says yes, human says no. That gap is how a miscalibrated grader gets caught.	

Rare, and the reason the line exists. Of 3,357 ledger rows, 38 carry both a machine verdict and a human one. 18 agree, 20 disagree, and 6 disagree in this direction: machine pass, human reject.

One row where the grader and I disagree measures calibration. Ten agreements do not.

Each field has a reader.
Not all read yet.

1. accepted feeds the preference log: every accept and reject stored as a pair, per skill and model
2. assertion_pass feeds reward mining, joined to accepted through the correlation_id
3. decided_model versus what ran feeds the scorecard of the part that recommends the model, once real observations exist
4. drift watch compares this week's rows against snapshot baselines

and the rulebook only unlocks the phrase self-improving when these consumers provably run

That is the design. The honest status is printed on the line itself: decision_mode says advisory, decision_real_n says 0. The wiring exists, the current is intermittent.

> **ASIDE.** My routing brain writes its homework down and does not yet read it. Which is precisely why the ban in rule #32 has not been lifted.

The ledger is complete. The reading of it is the unfinished part, and the line admits that.

One line per task is the cheapest thing here

39

daily files, one per day from 2026-06-27 to 2026-08-04, no missing dates.

Because every loop I want needs this exact substrate. You cannot calibrate a grader without machine grades joined to human verdicts. You cannot route by track record without a track record. You cannot detect drift without a baseline made of dated rows.

One line per task, and every expensive thing built since sits on top of it. Every claim I am allowed to make in public is bounded by what this file can prove.

> **REJECTED.** A dashboard-first analytics stack. The line came first; charts wait for consumers that actually exist.

You cannot show a loop improved without rows from before it changed.

Instrument what the work was worth, not only what it cost.

Both verdicts sit on one line, in a file the worker never writes itself. Full dissection and the commands to check it.

sgnk.ai/brilliant ↗

Sagnik Mitra

I build AI systems and the machinery that keeps them honest: gates, ledgers, judges, and rails. Product design for years, engineering the whole time. This series is what the building actually taught me.

sgnk.ai · in/sagnikmitra · github.com/sagnikmitra